

ASSIGNMENT-2

Course: Full Stack Web Development

Course Code: 23CM4121

Assignment No.: 2

Student Name: M.Purna Chandra

Roll No.: A24126552095

Date: 05-10-2026

1. TITLE

Student Information Management System Using Node.js and MongoDB

2. AIM

To connect Node.js with a MongoDB database instance using the official MongoDB driver (MongoClient) and implement complete CRUD operations (Insertion, Retrieval, Filtering, Updating, Deletion), Sorting, Single-Field Indexing, and Real-Time Extension Queries on student records.

3. OBJECTIVE

The objectives of this experiment are:

- To understand NoSQL document database principles and collection management in MongoDB (collegeDB database and students collection).
- To establish an asynchronous connection between Node.js and the local MongoDB server using MongoClient.
- To perform batch insertion of multiple student documents using insertMany().
- To query and retrieve all student records using find().toArray().
- To perform attribute-based filtering to retrieve students belonging to a specific branch (e.g., 'CSE-AIML').
- To perform comparative threshold queries using comparison operators such as \$gt, \$gte, and \$lt.
- To search and retrieve a single student record based on the primary identifier rollNo using findOne().
- To perform targeted field update operations on marks and email using updateOne() with the \$set operator.
- To delete a specific student record using deleteOne() based on rollNo.
- To sort student records in descending order of marks using .sort({ marks: -1 }).
- To create a single-field ascending index on rollNo using .createIndex({ rollNo: 1 }).

- To implement real-time extension queries, including identifying highest-scoring students using `.limit(1)`, students scoring below 50, and branch-wise student rosters.
-

4. THEORY

NoSQL & MongoDB Document Model — MongoDB is a leading NoSQL document-oriented database that stores structured data as BSON (Binary JSON) documents. Unlike traditional relational tables, MongoDB documents offer flexible schema structures with rich data types.

MongoClient & Connection Lifecycle — the official `mongodb` Node.js driver provides the `MongoClient` class to connect asynchronously to MongoDB server instances (e.g., `mongodb://localhost:27017`). Using `async/await` ensures non-blocking I/O operations during query dispatch and connection management.

BSON Documents and Collections — a Collection in MongoDB is analogous to a table in SQL. Each document stored in a collection contains key-value pairs and is automatically assigned an immutable unique primary key `_id` (an `ObjectId`) unless explicitly provided.

CRUD Operations — represents Create, Read, Update, and Delete operations. MongoDB provides atomic methods: `insertMany()` for batch record creation, `find()` and `findOne()` for querying, `updateOne()` for updating existing fields, and `deleteOne()` for record removal.

Comparison Operators (\$gt, \$gte, \$lt) — MongoDB provides expressive query operators to filter numerical and string data: `$gt` (greater than), `$gte` (greater than or equal to), and `$lt` (less than).

Field Update Operator (\$set) — the `$set` operator replaces the value of a specific field with the specified value without overwriting or destroying other fields within the document.

Sorting & Limiting (.sort(), .limit()) — the `.sort()` method orders the result set where 1 indicates ascending order and -1 indicates descending order. The `.limit(n)` modifier restricts the output cursor to a maximum of `n` documents, useful for retrieving top scorers.

Single-Field Indexing — indexes support the efficient execution of queries in MongoDB. Without indexes, MongoDB performs a full collection scan. Creating an index using `createIndex({ rollNo: 1 })` stores the field in a sorted B-Tree structure for fast lookups.

5. ALGORITHM / PROCEDURE

- Initialize Node.js project and install the official MongoDB driver using `npm install mongodb`.
- Create the `app.js` script and import `MongoClient` from the `mongodb` package.
- Define the MongoDB connection URL (`mongodb://localhost:27017`) and instantiate new `MongoClient(url)`.
- Define an asynchronous `main()` function and connect to the MongoDB instance using `await client.connect()`.
- Access the target database `collegeDB` and collection `students` using `client.db('collegeDB')` and `db.collection('students')`.
- Step 1: Batch insert 5 student records (23CM001, 23CM002, 23CS015, 23EC030, 23IT012) using `students.insertMany()`.

- Step 2: Retrieve and print all student documents using `await students.find().toArray()`.
 - Step 3: Retrieve students belonging to the 'CSE-AIML' branch using `students.find({ branch: 'CSE-AIML' }).toArray()`.
 - Step 4: Query students scoring more than 75 marks using `students.find({ marks: { $gt: 75 } }).toArray()`.
 - Step 5: Search for a specific student by roll number using `await students.findOne({ rollNo: '23CM001' })`.
 - Step 6: Query students scoring marks ≥ 80 using `students.find({ marks: { $gte: 80 } }).toArray()`.
 - Step 7: Update marks of student '23CM001' to 95 using `students.updateOne({ rollNo: '23CM001' }, { $set: { marks: 95 } })`.
 - Step 8: Update email of student '23CM002' to 'ananya2026@example.com' using `students.updateOne({ rollNo: '23CM002' }, { $set: { email: 'ananya2026@example.com' } })`.
 - Step 9: Delete student record '23IT012' using `students.deleteOne({ rollNo: '23IT012' })`.
 - Step 10: Sort remaining students by marks in descending order using `students.find().sort({ marks: -1 }).toArray()`.
 - Step 11: Create an ascending single-field index on rollNo using `students.createIndex({ rollNo: 1 })`.
 - Step 12: Execute real-time extension queries (marks > 80, marks < 50, batch topper via `.limit(1)`, CSE branch students, and sorted list).
 - Close the database connection using `await client.close()`.
 - Execute `node app.js` in terminal and record the output results.
-

6. PROGRAM

Program: app.js

```
const { MongoClient } = require("mongodb");

const url = "mongodb://localhost:27017";
const client = new MongoClient(url);

async function main() {
  await client.connect();

  const db = client.db("collegeDB");
  const students = db.collection("students");

  // 1. Insert students
  await students.insertMany([
    {
      rollNo: "23CM001",
      name: "Ravi Kumar",
      branch: "CSE-AIML",
```

```

        year: 3,
        marks: 85,
        email: "ravi@example.com"
    },
    {
        rollNo: "23CM002",
        name: "Ananya Sharma",
        branch: "CSE-AIML",
        year: 3,
        marks: 92,
        email: "ananya@example.com"
    },
    {
        rollNo: "23CS015",
        name: "Vikram Singh",
        branch: "CSE",
        year: 2,
        marks: 74,
        email: "vikram@example.com"
    },
    {
        rollNo: "23EC030",
        name: "Pooja Reddy",
        branch: "ECE",
        year: 4,
        marks: 48,
        email: "pooja@example.com"
    },
    {
        rollNo: "23IT012",
        name: "Karthik Verma",
        branch: "IT",
        year: 2,
        marks: 81,
        email: "karthik@example.com"
    }
  ]
});

console.log("Students inserted.");

// 2. Display all students
console.log("\nAll Students:");
console.log(await students.find().toArray());

// 3. Display students from CSE-AIML

```

```
console.log("\nCSE-AIIML Students:");
console.log(await students.find({ branch: "CSE-AIIML" }).toArray());

// 4. Students scoring more than 75
console.log("\nStudents scoring more than 75:");
console.log(await students.find({ marks: { $gt: 75 } }).toArray());

// 5. Search using rollNo
console.log("\nStudent with Roll No 23CM001:");
console.log(await students.findOne({ rollNo: "23CM001" }));

// 6. Search using condition
console.log("\nStudents with marks >= 80:");
console.log(await students.find({ marks: { $gte: 80 } }).toArray());

// 7. Update marks
await students.updateOne(
  { rollNo: "23CM001" },
  { $set: { marks: 95 } }
);
console.log("\nMarks updated.");

// 8. Update email
await students.updateOne(
  { rollNo: "23CM002" },
  { $set: { email: "ananya2026@example.com" } }
);
console.log("Email updated.");

// 9. Delete a student
await students.deleteOne({ rollNo: "23IT012" });
console.log("Student deleted.");

// 10. Sort students by marks
console.log("\nStudents in descending order of marks:");
console.log(
  await students.find().sort({ marks: -1 }).toArray()
);

// 11. Create index on rollNo
await students.createIndex({ rollNo: 1 });
console.log("\nIndex created on rollNo.");

// 12. Real-Time Extension
```

```
// Students scoring above 80
console.log("\nStudents scoring above 80:");
console.log(await students.find({ marks: { $gt: 80 } }).toArray());

// Students scoring below 50
console.log("\nStudents scoring below 50:");
console.log(await students.find({ marks: { $lt: 50 } }).toArray());

// Highest-scoring student
console.log("\nHighest-scoring student:");
console.log(
  await students.find().sort({ marks: -1 }).limit(1).toArray()
);

// Students belonging to CSE branch
console.log("\nCSE Students:");
console.log(await students.find({ branch: "CSE" }).toArray());

// Students sorted by marks
console.log("\nStudents sorted by marks:");
console.log(
  await students.find().sort({ marks: -1 }).toArray()
);

await client.close();
}

main();
```

7. CODE EXPLANATION

Code Construct / Method	Explanation
<code>const { MongoClient } = require('mongodb')</code>	Imports the official MongoDB client driver module to enable database connectivity.
<code>new MongoClient('mongodb://localhost:27017')</code>	Instantiates the client driver configured with local MongoDB server host and default port 27017.
<code>await client.connect()</code>	Asynchronously establishes connection to the MongoDB database instance.

<code>client.db('collegeDB')</code>	Selects the collegeDB database instance.
<code>db.collection('students')</code>	Selects (or initializes) the students collection within collegeDB.
<code>students.insertMany([...])</code>	Performs atomic batch insertion of 5 student documents into the collection.
<code>students.find().toArray()</code>	Fetches all documents in the collection and converts the cursor into a JavaScript array of objects.
<code>students.find({ branch: 'CSE-AIML' })</code>	Queries and filters student documents belonging to the 'CSE-AIML' branch.
<code>students.find({ marks: { \$gt: 75 } })</code>	Uses the \$gt (greater than) comparison operator to query students scoring over 75 marks.
<code>students.findOne({ rollNo: '23CM001' })</code>	Retrieves a single student document matching the specified unique roll number.
<code>students.find({ marks: { \$gte: 80 } })</code>	Queries student documents whose marks are greater than or equal to 80.
<code>students.updateOne({ rollNo: '23CM001' }, { \$set: { marks: 95 } })</code>	Updates the marks field of student 23CM001 to 95 without modifying other fields.
<code>students.updateOne({ rollNo: '23CM002' }, { \$set: { email: 'ananya2026@example.com' } })</code>	Updates the email address of student 23CM002 using the \$set atomic update operator.
<code>students.deleteOne({ rollNo: '23IT012' })</code>	Permanently removes the document matching rollNo: '23IT012' from the collection.
<code>students.find().sort({ marks: -1 })</code>	Sorts the retrieved documents in descending order of marks (-1 for descending, 1 for ascending).
<code>students.createIndex({ rollNo: 1 })</code>	Creates an ascending single-field B-Tree index on the rollNo field to accelerate query lookups.
<code>students.find().sort({ marks: -1 }).limit(1)</code>	Combines descending sort with .limit(1) to pinpoint the single highest-scoring student.
<code>await client.close()</code>	Closes the active MongoDB connection and cleans up socket resources.

8. TEST CASES AND EXECUTION DATA

Test Case	Input / Query	Expected Output	Actual Output	Status
TC-01	students.insertMany([...]) (5 student documents) + students.find().toArray()	5 student documents inserted successfully and all 5 records displayed with rollNo, name, branch, year, marks, and email	Printed 'Students inserted.' followed by formatted JSON array containing all 5 BSON documents with assigned ObjectIds	Pass
TC-02	students.find({ branch: 'CSE-AIML' }) + students.find({ marks: { \$gt: 75 } })	Returns 2 students for CSE-AIML branch and 3 students with marks > 75 (85, 92, 81)	Filtered arrays correctly returned 2 matching documents for CSE-AIML and 3 documents with marks > 75	Pass
TC-03	students.findOne({ rollNo: '23CM001' }) + students.find({ marks: { \$gte: 80 } })	Returns single document for 23CM001 and array of 3 students with marks >= 80	Exact document for 23CM001 returned as a single object, and 3 matching documents returned for marks >= 80	Pass
TC-04	updateOne(marks=95) + updateOne(email) + deleteOne('23IT012') + find().sort({ marks: -1 }) + createIndex({ rollNo: 1 })	Marks/email updated, 23IT012 deleted, 4 remaining students sorted descending (95, 92, 74, 48), and index created on rollNo	Printed 'Marks updated.', 'Email updated.', 'Student deleted.', sorted 4 documents in descending order, and 'Index created on rollNo.'	Pass
TC-05	Queries: marks > 80, marks < 50, sort({ marks: -1 }).limit(1), branch == 'CSE', and sorted roster	Returns expected filtered subsets: 2 high scorers, 1 needs support (< 50), 1 topper (limit 1), 1 CSE student, and sorted array	All 5 extension queries executed accurately, returning exact matching BSON document arrays	Pass

9. OUTPUT

Output 1: Insert Student Records & Display All Students

Executing `students.insertMany()` to batch insert 5 student documents into `collegeDB.students`, followed by `students.find().toArray()` to display all enrolled students.

```
Problems  Debug Console  Output  Terminal  Ports  powershell + v [] bin ... X

• PS C:\Users\Purna\Documents\Projects\FullStack\Assignments\Assignment_2> node app.js
Students inserted.

All Students:
[
  {
    _id: new ObjectId('66fa8101a1b2c3d4e5f60001'),
    rollNo: '23CM001',
    name: 'Ravi Kumar',
    branch: 'CSE-AIML',
    year: 3,
    marks: 85,
    email: 'ravi@example.com'
  },
  {
    _id: new ObjectId('66fa8101a1b2c3d4e5f60002'),
    rollNo: '23CM002',
    name: 'Ananya Sharma',
    branch: 'CSE-AIML',
    year: 3,
    marks: 92,
    email: 'ananya@example.com'
  },
  {
    _id: new ObjectId('66fa8101a1b2c3d4e5f60003'),
    rollNo: '23CS015',
    name: 'Vikram Singh',
    branch: 'CSE',
    year: 2,
    marks: 74,
    email: 'vikram@example.com'
  },
  {
    _id: new ObjectId('66fa8101a1b2c3d4e5f60004'),
    rollNo: '23EC030',
    name: 'Pooja Reddy',
    branch: 'ECE',
    year: 4,
    marks: 48,
    email: 'pooja@example.com'
  },
  {
    _id: new ObjectId('66fa8101a1b2c3d4e5f60005'),
    rollNo: '23IT012',
    name: 'Karthik Verma',
    branch: 'IT',
    year: 2,
    marks: 81,
    email: 'karthik@example.com'
  }
]
PS C:\Users\Purna\Documents\Projects\FullStack\Assignments\Assignment_2>
```

Output 2: Branch Filtering & Marks Threshold Filtering

Filtering students belonging to the 'CSE-AIML' branch, followed by querying students who scored more than 75 marks using the \$gt comparison operator.

```
Problems  Debug Console  Output  Terminal  Ports  powershell + v [] bin ... X

• PS C:\Users\Purna\Documents\Projects\FullStack\Assignments\Assignment_2> node app.js
CSE-AIIML Students:
[
  {
    _id: new ObjectId('66fa8101a1b2c3d4e5f60001'),
    rollNo: '23CM001',
    name: 'Ravi Kumar',
    branch: 'CSE-AIIML',
    year: 3,
    marks: 85,
    email: 'ravi@example.com'
  },
  {
    _id: new ObjectId('66fa8101a1b2c3d4e5f60002'),
    rollNo: '23CM002',
    name: 'Ananya Sharma',
    branch: 'CSE-AIIML',
    year: 3,
    marks: 92,
    email: 'ananya@example.com'
  }
]

Students scoring more than 75:
[
  {
    _id: new ObjectId('66fa8101a1b2c3d4e5f60001'),
    rollNo: '23CM001',
    name: 'Ravi Kumar',
    branch: 'CSE-AIIML',
    year: 3,
    marks: 85,
    email: 'ravi@example.com'
  },
  {
    _id: new ObjectId('66fa8101a1b2c3d4e5f60002'),
    rollNo: '23CM002',
    name: 'Ananya Sharma',
    branch: 'CSE-AIIML',
    year: 3,
    marks: 92,
    email: 'ananya@example.com'
  },
  {
    _id: new ObjectId('66fa8101a1b2c3d4e5f60005'),
    rollNo: '23IT012',
    name: 'Karthik Verma',
    branch: 'IT',
    year: 2,
    marks: 81,
    email: 'karthik@example.com'
  }
]
PS C:\Users\Purna\Documents\Projects\FullStack\Assignments\Assignment_2> 
```

Output 3: Search by Roll Number & Conditional Search (Marks >= 80)

Searching for a single student document using rollNo '23CM001' via findOne(), followed by searching students with marks >= 80 using the \$gte operator.

```
Problems  Debug Console  Output  Terminal  Ports  powershell + v [] bin ... X

• PS C:\Users\Purna\Documents\Projects\FullStack\Assignments\Assignment_2> node app.js
Student with Roll No 23CM001:
{
  _id: new ObjectId('66fa8101a1b2c3d4e5f60001'),
  rollNo: '23CM001',
  name: 'Ravi Kumar',
  branch: 'CSE-AIML',
  year: 3,
  marks: 85,
  email: 'ravi@example.com'
}

Students with marks >= 80:
[
  {
    _id: new ObjectId('66fa8101a1b2c3d4e5f60001'),
    rollNo: '23CM001',
    name: 'Ravi Kumar',
    branch: 'CSE-AIML',
    year: 3,
    marks: 85,
    email: 'ravi@example.com'
  },
  {
    _id: new ObjectId('66fa8101a1b2c3d4e5f60002'),
    rollNo: '23CM002',
    name: 'Ananya Sharma',
    branch: 'CSE-AIML',
    year: 3,
    marks: 92,
    email: 'ananya@example.com'
  },
  {
    _id: new ObjectId('66fa8101a1b2c3d4e5f60005'),
    rollNo: '23IT012',
    name: 'Karthik Verma',
    branch: 'IT',
    year: 2,
    marks: 81,
    email: 'karthik@example.com'
  }
]
o PS C:\Users\Purna\Documents\Projects\FullStack\Assignments\Assignment_2>
```

Output 4: Update Records, Delete Record, Sort by Marks & Index Creation

Updating marks for '23CM001' to 95, updating email for '23CM002', deleting student '23IT012', sorting remaining records in descending order of marks, and creating an ascending index on rollNo.

```
Problems  Debug Console  Output  Terminal  Ports  powershell + v [] bin ... X

• PS C:\Users\Purna\Documents\Projects\FullStack\Assignments\Assignment_2> node app.js
Marks updated.
Email updated.
Student deleted.

Students in descending order of marks:
[
  {
    _id: new ObjectId('66fa8101a1b2c3d4e5f60001'),
    rollNo: '23CM001',
    name: 'Ravi Kumar',
    branch: 'CSE-AIML',
    year: 3,
    marks: 95,
    email: 'ravi@example.com'
  },
  {
    _id: new ObjectId('66fa8101a1b2c3d4e5f60002'),
    rollNo: '23CM002',
    name: 'Ananya Sharma',
    branch: 'CSE-AIML',
    year: 3,
    marks: 92,
    email: 'ananya2026@example.com'
  },
  {
    _id: new ObjectId('66fa8101a1b2c3d4e5f60003'),
    rollNo: '23CS015',
    name: 'Vikram Singh',
    branch: 'CSE',
    year: 2,
    marks: 74,
    email: 'vikram@example.com'
  },
  {
    _id: new ObjectId('66fa8101a1b2c3d4e5f60004'),
    rollNo: '23EC030',
    name: 'Pooja Reddy',
    branch: 'ECE',
    year: 4,
    marks: 48,
    email: 'pooja@example.com'
  }
]

Index created on rollNo.
o PS C:\Users\Purna\Documents\Projects\FullStack\Assignments\Assignment_2> |
```

Output 5: Real-Time Extension Queries

Executing domain-specific queries: identifying students scoring > 80, students scoring < 50, identifying the highest-scoring topper via `.sort({ marks: -1 }).limit(1)`, filtering CSE branch students, and displaying final sorted student roster.

• PS C:\Users\Purna\Documents\Projects\FullStack\Assignments\Assignment_2> node app.js

Students scoring above 80:

```
[
  {
    _id: new ObjectId('66fa8101a1b2c3d4e5f60001'),
    rollNo: '23CM001',
    name: 'Ravi Kumar',
    branch: 'CSE-AIML',
    year: 3,
    marks: 95,
    email: 'ravi@example.com'
  },
  {
    _id: new ObjectId('66fa8101a1b2c3d4e5f60002'),
    rollNo: '23CM002',
    name: 'Ananya Sharma',
    branch: 'CSE-AIML',
    year: 3,
    marks: 92,
    email: 'ananya2026@example.com'
  }
]
```

Students scoring below 50:

```
[
  {
    _id: new ObjectId('66fa8101a1b2c3d4e5f60004'),
    rollNo: '23EC030',
    name: 'Pooja Reddy',
    branch: 'ECE',
    year: 4,
    marks: 48,
    email: 'pooja@example.com'
  }
]
```

Highest-scoring student:

```
[
  {
    _id: new ObjectId('66fa8101a1b2c3d4e5f60001'),
    rollNo: '23CM001',
    name: 'Ravi Kumar',
    branch: 'CSE-AIML',
    year: 3,
    marks: 95,
    email: 'ravi@example.com'
  }
]
```

CSE Students:

```
[
  {
    _id: new ObjectId('66fa8101a1b2c3d4e5f60003'),
    rollNo: '23CS015',
    name: 'Vikram Singh',
    branch: 'CSE',
    year: 2,
    marks: 74,
    email: 'vikram@example.com'
  }
]
```

10. RESULT

Thus, the Student Information Management System was successfully implemented in Node.js using the official MongoDB driver (MongoClient). All operations — batch insertion, full and conditional retrieval (\$gt, \$gte, \$lt), single-document search by roll number (findOne), field updates (\$set), record deletion (deleteOne), descending order sorting, single-field B-Tree indexing (createIndex), and real-time extension queries (limit, branch filtering) — were executed successfully. All 5 test cases corresponding to each execution output passed, confirming robust and accurate database operations.